

diffusion_model_run

February 8, 2026

1 Diffusion Model Evaluation (Test-Only)

This notebook is a **clean, reproducible evaluation script**. It performs **no training** and **no tuning**.

It answers three questions: 1. How good is the model on **one-step prediction**? 2. How does it behave under **rollout**? 3. How does performance differ on **transitions vs non-transitions** (and across phases)?

1.1 0) Inputs and invariants

These values are fixed for evaluation. Do **not** tune them here.

```
[1]: from pathlib import Path
import os, math, random, glob, re
import numpy as np
import pandas as pd
from PIL import Image
import matplotlib.pyplot as plt
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.nn.functional as F
from diffusers import UNet2DConditionModel, DDPMScheduler, DDIMScheduler

# Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', device)

# Project paths
```

```

PROJECT_ROOT = Path.cwd().resolve()
if PROJECT_ROOT.name != 'diffusion' and (PROJECT_ROOT / 'diffusion').exists():
    PROJECT_ROOT = PROJECT_ROOT / 'diffusion'
DATA_ROOT = PROJECT_ROOT.parent / 'data'
SILVER_ROOT = DATA_ROOT / 'embryo_dataset_silver'

# Model checkpoint (trained already)
MODEL_DIR = PROJECT_ROOT / 'embryo_diffusion_models'
DEFAULT_CKPT = MODEL_DIR / 'nextframe_ctxk7_128px_best.pt'
if DEFAULT_CKPT.exists():
    MODEL_CKPT_PATH = DEFAULT_CKPT
else:
    # Fallback: pick the most recent *_best.pt checkpoint if present
    best_ckpts = sorted(MODEL_DIR.glob('*_best.pt'), key=lambda p: p.stat().
        ↪st_mtime, reverse=True)
    MODEL_CKPT_PATH = best_ckpts[0] if best_ckpts else MODEL_DIR /
        ↪'nextframe_ctxk7_128px.pt'

# Evaluation limits (keep small for reproducibility/quick runs)
EVAL_MAX_SAMPLES = 256
ROLLOUT_SAMPLES = 12
ROLLOUT_STEPS = 7 # typically equal to CONTEXT_K; will be overwritten from
    ↪checkpoint if present
DDIM_STEPS = 20
DDIM_ETA = 0.0

# Artifacts
ARTIFACT_DIR = PROJECT_ROOT / 'diffusion_samples' / 'eval_artifacts'
ARTIFACT_DIR.mkdir(parents=True, exist_ok=True)
print('Artifacts:', ARTIFACT_DIR)

```

Device: cuda

Artifacts:

/home/khasion/Projects/thesis/diffusion/diffusion_samples/eval_artifacts

1.2 1) Load model + scheduler exactly as trained

We load the checkpoint config and recreate the UNet + diffusion schedulers with the same settings.

```

[2]: # --- Load checkpoint ---
assert MODEL_CKPT_PATH.exists(), f"Checkpoint not found: {MODEL_CKPT_PATH}"
ckpt = torch.load(MODEL_CKPT_PATH, map_location='cpu')
ckpt_cfg = ckpt.get('config', {})
print('Checkpoint config:', ckpt_cfg)

# Training-time config (fallbacks if missing)
IMG_SIZE = int(ckpt_cfg.get('IMG_SIZE', 128))
TIMESTEPS = int(ckpt_cfg.get('TIMESTEPS', 1000))

```

```

CONTEXT_K = int(ckpt_cfg.get('CONTEXT_K', 7))
FOCAL_PLANES = list(ckpt_cfg.get('FOCAL_PLANES', ['FO']))
NUM_PLANES = int(ckpt_cfg.get('NUM_PLANES', len(FOCAL_PLANES)))
DDIM_STEPS = int(ckpt_cfg.get('DDIM_STEPS', DDIM_STEPS))
DDIM_ETA = float(ckpt_cfg.get('DDIM_ETA', DDIM_ETA))

# These were hard-coded in training (keep consistent)
USE_TIME_MAP = True
MAX_FRAME_INDEX = 500
TIME_EMB_DIM = int(ckpt_cfg.get('TIME_EMB_DIM', 8))
CONTEXT_EMB_DIM = int(ckpt_cfg.get('CONTEXT_EMB_DIM', 64))
CTX_GRID = int(ckpt_cfg.get('CTX_GRID', 32))
BETA_START, BETA_END = 1e-4, 2e-2

print(f"IMG_SIZE={IMG_SIZE}, CONTEXT_K={CONTEXT_K}, TIMESTEPS={TIMESTEPS},  

    ↪ DDIM_STEPS={DDIM_STEPS}")
print('FOCAL_PLANES:', FOCAL_PLANES)

# --- Model definition (must match training) ---
class DiffusionCond(nn.Module):
    def __init__(self, context_k: int = CONTEXT_K, use_time_map: bool =  

    ↪ USE_TIME_MAP):
        super().__init__()
        self.context_k = context_k
        self.use_time_map = use_time_map
        self.context_emb_dim = CONTEXT_EMB_DIM
        self.ctx_grid = CTX_GRID

        # Learned absolute frame index embedding (time map)
        if self.use_time_map:
            self.frame_emb = nn.Embedding(MAX_FRAME_INDEX + 1, TIME_EMB_DIM)
            self.frame_proj = nn.Linear(TIME_EMB_DIM, 1)
        else:
            self.frame_emb = None
            self.frame_proj = None

        # Context encoder: per-frame tokens + learned positional embeddings
        self.ctx_encoder = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, stride=2, padding=1),
            nn.GELU(),
            nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
            nn.GELU(),
            nn.Conv2d(64, self.context_emb_dim, kernel_size=3, stride=1,  

    ↪ padding=1),
            nn.GELU(),
        )

```

```

        # Tiny temporal conv to summarize k context frames into a single
        ↪image-like channel
        self.ctx_to_img = nn.Conv2d(context_k, 1, kernel_size=1)

        self.ctx_pos_emb = nn.Embedding(context_k, self.context_emb_dim)
        self.ctx_spatial_pos_emb = nn.Parameter(torch.zeros(1, self.
        ↪context_emb_dim, self.ctx_grid, self.ctx_grid))
        self.ctx_time_emb = nn.Embedding(MAX_FRAME_INDEX + 1, self.
        ↪context_emb_dim)
        nn.init.trunc_normal_(self.ctx_spatial_pos_emb, std=0.02)

        # UNet2DConditionModel (cross-attention)
        in_ch = 1 + 1 + (1 if use_time_map else 0)
        self.unet = UNet2DConditionModel(
            sample_size=IMG_SIZE,
            in_channels=in_ch,
            out_channels=1,
            layers_per_block=2,
            block_out_channels=(16, 32, 64, 128),
            down_block_types=("DownBlock2D", "DownBlock2D",
        ↪"CrossAttnDownBlock2D", "CrossAttnDownBlock2D"),
            up_block_types=("CrossAttnUpBlock2D", "CrossAttnUpBlock2D",
        ↪"UpBlock2D", "UpBlock2D"),
            mid_block_type="UNetMidBlock2DCrossAttn",
            cross_attention_dim=self.context_emb_dim,
            norm_num_groups=16,
        )

        def build_time_map(self, frame_idx: torch.Tensor | int, h: int, w: int) ->
        ↪torch.Tensor:
            if not self.use_time_map:
                raise ValueError("build_time_map called but USE_TIME_MAP=False")
            if not torch.is_tensor(frame_idx):
                frame_idx = torch.tensor(frame_idx, device=next(self.parameters()).
        ↪device, dtype=torch.long)
            if frame_idx.dim() == 0:
                frame_idx = frame_idx.view(1)
            frame_idx = frame_idx.clamp(0, MAX_FRAME_INDEX).long()
            emb = self.frame_emb(frame_idx)
            scalar = self.frame_proj(emb)
            return scalar.view(-1, 1, 1, 1).expand(-1, 1, h, w)

        def encode_context(self, x_ctx: torch.Tensor, frame_idx: torch.Tensor |
        ↪None = None) -> torch.Tensor:
            if x_ctx.dim() == 3:
                x_ctx = x_ctx.unsqueeze(0)
            if x_ctx.shape[1] != self.context_k:

```

```

        raise ValueError(f"Expected context with k={self.context_k}␣
↪channels, got {x_ctx.shape[1]}")
        b, k, h, w = x_ctx.shape
        x = x_ctx.reshape(b * k, 1, h, w)
        x = self.ctx_encoder(x)
        if x.shape[-2:] != (self.ctx_grid, self.ctx_grid):
            x = F.interpolate(x, size=(self.ctx_grid, self.ctx_grid),␣
↪mode="bilinear", align_corners=False)
        x = x + self.ctx_spatial_pos_emb
        x = x.view(b, k, self.context_emb_dim, self.ctx_grid, self.ctx_grid)
        pos = self.ctx_pos_emb(torch.arange(k, device=x_ctx.device))
        x = x + pos.view(1, k, self.context_emb_dim, 1, 1)
        if frame_idx is not None:
            if not torch.is_tensor(frame_idx):
                frame_idx = torch.tensor(frame_idx, device=x_ctx.device,␣
↪dtype=torch.long)
            if frame_idx.dim() == 0:
                frame_idx = frame_idx.view(1)
            frame_idx = frame_idx.clamp(0, MAX_FRAME_INDEX).long()
            t_end = (frame_idx - 1).clamp(0, MAX_FRAME_INDEX)
            offsets = torch.arange(k - 1, -1, -1, device=x_ctx.device)
            ctx_idx = (t_end.unsqueeze(1) - offsets.unsqueeze(0)).clamp(0,␣
↪MAX_FRAME_INDEX)
            ctx_time = self.ctx_time_emb(ctx_idx)
            x = x + ctx_time.view(b, k, self.context_emb_dim, 1, 1)
            x = x.permute(0, 1, 3, 4, 2).contiguous()
            x = x.view(b, k * self.ctx_grid * self.ctx_grid, self.context_emb_dim)
            return x
        def build_input(self, x_noisy: torch.Tensor, x_ctx: torch.Tensor, time_map:␣
↪torch.Tensor | None = None) -> torch.Tensor:
            if x_ctx.dim() == 3:
                x_ctx = x_ctx.unsqueeze(0)
            ctx_img = self.ctx_to_img(x_ctx)
            if self.use_time_map:
                if time_map is None:
                    raise ValueError("time_map is required when USE_TIME_MAP=True")
                return torch.cat([x_noisy, ctx_img, time_map], dim=1)
            return torch.cat([x_noisy, ctx_img], dim=1)
        def forward(self, x_noisy, t, x_ctx, time_map: torch.Tensor | None = None,␣
↪frame_idx: torch.Tensor | None = None):
            x_in = self.build_input(x_noisy, x_ctx, time_map=time_map)
            enc = self.encode_context(x_ctx, frame_idx=frame_idx)
            return self.unet(x_in, t, encoder_hidden_states=enc).sample

```

Instantiate and load weights

```

diffusion = DiffusionCond(context_k=CONTEXT_K, use_time_map=USE_TIME_MAP).
    ↪to(device)
diffusion.load_state_dict(ckpt['model'], strict=True)
diffusion.eval()
print('Loaded diffusion model.')

# Schedulers (match training)
ddpm_scheduler = DDPMScheduler(
    num_train_timesteps=TIMESTEPS,
    beta_start=BETA_START,
    beta_end=BETA_END,
    beta_schedule='scaled_linear',
    variance_type='fixed_small',
)
ddim_scheduler = DDIMScheduler(
    num_train_timesteps=TIMESTEPS,
    beta_start=BETA_START,
    beta_end=BETA_END,
    beta_schedule='scaled_linear',
    clip_sample=False,
    set_alpha_to_one=False,
)

# Sampling helpers
def _ensure_context_batch(x_ctx: torch.Tensor | None, shape):
    B, H, W = shape[0], shape[2], shape[3]
    if x_ctx is None:
        return torch.zeros((B, CONTEXT_K, H, W), device=device)
    x_ctx = x_ctx.to(device)
    if x_ctx.dim() == 3:
        x_ctx = x_ctx.unsqueeze(0)
    if x_ctx.shape[1] != CONTEXT_K:
        raise ValueError(f"Expected context with k={CONTEXT_K} channels, got_
    ↪{x_ctx.shape[1]}")
    return x_ctx

def _ensure_time_map(frame_idx: torch.Tensor | int | None, shape):
    if not USE_TIME_MAP:
        return None
    if frame_idx is None:
        raise ValueError("frame_idx is required when USE_TIME_MAP=True")
    if not torch.is_tensor(frame_idx):
        frame_idx = torch.tensor(frame_idx, device=device, dtype=torch.long)
    if frame_idx.dim() == 0:
        frame_idx = frame_idx.view(1)
    frame_idx = frame_idx.clamp(0, MAX_FRAME_INDEX).long()
    return diffusion.build_time_map(frame_idx, shape[2], shape[3])

```

```

def _eps_from_model(x_noisy: torch.Tensor, x_ctx: torch.Tensor, t_tensor: torch.
    ↪Tensor, time_map: torch.Tensor | None = None, frame_idx: torch.Tensor | None
    ↪= None):
    return diffusion.forward(x_noisy, t_tensor, x_ctx, time_map=time_map,
    ↪frame_idx=frame_idx)

@torch.no_grad()
def sample_from_model(shape, x_ctx, frame_idx=None, steps=DDIM_STEPS,
    ↪eta=DDIM_ETA):
    B = shape[0]
    x = torch.randn((B, 1, shape[2], shape[3]), device=device)
    x_ctx = _ensure_context_batch(x_ctx, shape)
    time_map = _ensure_time_map(frame_idx, shape)
    ddim_scheduler.set_timesteps(steps, device=device)
    for t in ddim_scheduler.timesteps:
        t_tensor = torch.full((B,), int(t), device=device, dtype=torch.long)
        eps = _eps_from_model(x, x_ctx, t_tensor, time_map=time_map,
    ↪frame_idx=frame_idx)
        x = ddim_scheduler.step(eps, t, x, eta=eta).prev_sample
    return x.detach()

```

Checkpoint config: {'IMG_SIZE': 128, 'TIMESTEPS': 600, 'PHASE_LABELS': ['pPB2', 'pPNa', 'pPNf', 'p2', 'p3', 'p4', 'p5', 'p6', 'p7', 'p8', 'p9+', 'pM', 'pSB', 'pB', 'pEB', 'pHB'], 'FOCAL_PLANES': ['F0'], 'NUM_PLANES': 1, 'DDIM_STEPS': 150, 'DDIM_ETA': 0.0, 'CONTEXT_K': 7}

IMG_SIZE=128, CONTEXT_K=7, TIMESTEPS=600, DDIM_STEPS=150

FOCAL_PLANES: ['F0']

Checkpoint config: {'IMG_SIZE': 128, 'TIMESTEPS': 600, 'PHASE_LABELS': ['pPB2', 'pPNa', 'pPNf', 'p2', 'p3', 'p4', 'p5', 'p6', 'p7', 'p8', 'p9+', 'pM', 'pSB', 'pB', 'pEB', 'pHB'], 'FOCAL_PLANES': ['F0'], 'NUM_PLANES': 1, 'DDIM_STEPS': 150, 'DDIM_ETA': 0.0, 'CONTEXT_K': 7}

IMG_SIZE=128, CONTEXT_K=7, TIMESTEPS=600, DDIM_STEPS=150

FOCAL_PLANES: ['F0']

```

-----
RuntimeError                                Traceback (most recent call last)
Cell In[2], line 130
    128 # Instantiate and load weights
    129 diffusion = DiffusionCond(context_k=CONTEXT_K,
    ↪use_time_map=USE_TIME_MAP).to(device)
--> 130 diffusion.load_state_dict(ckpt['model'], strict=True)
    131 diffusion.eval()
    132 print('Loaded diffusion model.')

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
    ↪module.py:2629, in Module.load_state_dict(self, state_dict, strict, assign)
    2621         error_msgs.insert(

```

```

2622         0,
2623         "Missing key(s) in state_dict: {}".format(
2624             ", ".join(f'"{k}"' for k in missing_keys)
2625         ),
2626     )
2628 if len(error_msgs) > 0:
-> 2629     raise RuntimeError(
2630         "Error(s) in loading state_dict for {}: \n\t{}".format(
2631             self.__class__.__name__, "\n\t".join(error_msgs)
2632         )
2633     )
2634 return _IncompatibleKeys(missing_keys, unexpected_keys)

```

```

RuntimeError: Error(s) in loading state_dict for DiffusionCond:

```

Missing key(s) in state_dict: "ctx_spatial_pos_emb", "frame_emb.weight",
↪ "frame_proj.weight", "frame_proj.bias", "ctx_encoder.0.weight", "ctx_encoder.0.
↪ bias", "ctx_encoder.2.weight", "ctx_encoder.2.bias", "ctx_encoder.4.weight",
↪ "ctx_encoder.4.bias", "ctx_to_img.weight", "ctx_to_img.bias", "ctx_pos_emb.
↪ weight", "ctx_time_emb.weight", "unet.conv_in.weight", "unet.conv_in.bias",
↪ "unet.time_embedding.linear_1.weight", "unet.time_embedding.linear_1.bias",
↪ "unet.time_embedding.linear_2.weight", "unet.time_embedding.linear_2.bias",
↪ "unet.down_blocks.0.resnets.0.norm1.weight", "unet.down_blocks.0.resnets.0.
↪ norm1.bias", "unet.down_blocks.0.resnets.0.conv1.weight", "unet.down_blocks.0.
↪ resnets.0.conv1.bias", "unet.down_blocks.0.resnets.0.time_emb_proj.weight",
↪ "unet.down_blocks.0.resnets.0.time_emb_proj.bias", "unet.down_blocks.0.resnet
↪ 0.norm2.weight", "unet.down_blocks.0.resnets.0.norm2.bias", "unet.down_blocks
↪ 0.resnets.0.conv2.weight", "unet.down_blocks.0.resnets.0.conv2.bias", "unet.
↪ down_blocks.0.resnets.1.norm1.weight", "unet.down_blocks.0.resnets.1.norm1.
↪ bias", "unet.down_blocks.0.resnets.1.conv1.weight", "unet.down_blocks.0.
↪ resnets.1.conv1.bias", "unet.down_blocks.0.resnets.1.time_emb_proj.weight",
↪ "unet.down_blocks.0.resnets.1.time_emb_proj.bias", "unet.down_blocks.0.resnet
↪ 1.norm2.weight", "unet.down_blocks.0.resnets.1.norm2.bias", "unet.down_blocks
↪ 0.resnets.1.conv2.weight", "unet.down_blocks.0.resnets.1.conv2.bias", "unet.
↪ down_blocks.0.downsamplers.0.conv.weight", "unet.down_blocks.0.downsamplers.0.
↪ conv.bias", "unet.down_blocks.1.resnets.0.norm1.weight", "unet.down_blocks.1.
↪ resnets.0.norm1.bias", "unet.down_blocks.1.resnets.0.conv1.weight", "unet.
↪ down_blocks.1.resnets.0.conv1.bias", "unet.down_blocks.1.resnets.0.
↪ time_emb_proj.weight", "unet.down_blocks.1.resnets.0.time_emb_proj.bias",
↪ "unet.down_blocks.1.resnets.0.norm2.weight", "unet.down_blocks.1.resnets.0.
↪ norm2.bias", "unet.down_blocks.1.resnets.0.conv2.weight", "unet.down_blocks.1
↪ resnets.0.conv2.bias", "unet.down_blocks.1.resnets.0.conv_shortcut.weight",
↪ "unet.down_blocks.1.resnets.0.conv_shortcut.bias", "unet.down_blocks.1.resnet
↪ 1.norm1.weight", "unet.down_blocks.1.resnets.1.norm1.bias", "unet.down_blocks
↪ 1.resnets.1.conv1.weight", "unet.down_blocks.1.resnets.1.conv1.bias", "unet.
↪ down_blocks.1.resnets.1.time_emb_proj.weight", "unet.down_blocks.1.resnets.1.
↪ time_emb_proj.bias", "unet.down_blocks.1.resnets.1.norm2.weight", "unet.
↪ down_blocks.1.resnets.1.norm2.bias", "unet.down_blocks.1.resnets.1.conv2.
↪ weight", "unet.down_blocks.1.resnets.1.conv2.bias", "unet.down_blocks.1.
↪ downsamplers.0.conv.weight", "unet.down_blocks.1.downsamplers.0.conv.bias",
↪ "unet.down_blocks.2.attentions.0.norm.weight", "unet.down_blocks.2.attentions
↪ 0.norm.bias", "unet.down_blocks.2.attentions.0.proj_in.weight", "unet.
↪ down_blocks.2.attentions.0.proj_in.bias", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.norm1.weight", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.norm1.bias", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.attn1.to_q.weight", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.attn1.to_k.weight", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.attn1.to_v.weight", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.attn1.to_out.0.weight", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.attn1.to_out.0.bias", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.norm2.weight", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.norm2.bias", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.attn2.to_q.weight", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.attn2.to_k.weight", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.attn2.to_v.weight", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.attn2.to_out.0.weight", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.attn2.to_out.0.bias", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.norm3.weight", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.norm3.bias", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.ff.net.0.proj.weight", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.ff.net.0.proj.bias", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.ff.net.2.weight", "unet.down_blocks.2.attentions.0.
↪ transformer_blocks.0.ff.net.2.bias", "unet.down_blocks.2.attentions.0.proj_out
↪ weight", "unet.down_blocks.2.attentions.0.proj_out.bias", "unet.down_blocks.2
↪ attentions.1.norm.weight", "unet.down_blocks.2.attentions.1.norm.bias", "unet
↪ down_blocks.2.attentions.1.proj_in.weight", "unet.down_blocks.2.attentions.1.
↪ proj_in.bias", "unet.down_blocks.2.attentions.1.transformer_blocks.0.norm1.
↪ weight", "unet.down_blocks.2.attentions.1.transformer_blocks.0.norm1.bias",
↪ "unet.down_blocks.2.attentions.1.transformer_blocks.0.attn1.to_q.weight",
↪ "unet.down_blocks.2.attentions.1.transformer_blocks.0.attn1.to_k.weight",
↪ "unet.down_blocks.2.attentions.1.transformer_blocks.0.attn1.to_v.weight",
↪ "unet.down_blocks.2.attentions.1.transformer_blocks.0.attn1.to_out.0.weight",
↪ "unet.down_blocks.2.attentions.1.transformer_blocks.0.attn1.to_out.0.bias",
↪ "unet.down_blocks.2.attentions.1.transformer_blocks.0.norm2.weight", "unet.
↪ down_blocks.2.attentions.1.transformer_blocks.0.norm2.bias", "unet.down_block
↪ 2.attentions.1.transformer_blocks.0.attn2.to_q.weight", "unet.down_blocks.2.
↪ attentions.1.transformer_blocks.0.attn2.to_k.weight", "unet.down_blocks.2.
↪ attentions.1.transformer_blocks.0.attn2.to_v.weight", "unet.down_blocks.2.
↪ attentions.1.transformer_blocks.0.attn2.to_out.0.weight", "unet.down_blocks.2.
↪ attentions.1.transformer_blocks.0.attn2.to_out.0.bias", "unet.down_blocks.2.
↪ attentions.1.transformer_blocks.0.norm3.weight", "unet.down_blocks.2.
↪ attentions.1.transformer_blocks.0.norm3.bias", "unet.down_blocks.2.attentions
↪ 1.transformer_blocks.0.ff.net.0.proj.weight", "unet.down_blocks.2.attentions...

```
Unexpected key(s) in state_dict: "_orig_mod.ctx_spatial_pos_emb",  
↪ "_orig_mod.frame_emb.weight", "_orig_mod.frame_proj.weight", "_orig_mod.  
↪ frame_proj.bias", "_orig_mod.ctx_encoder.0.weight", "_orig_mod.ctx_encoder.0.  
↪ bias", "_orig_mod.ctx_encoder.2.weight", "_orig_mod.ctx_encoder.2.bias",  
↪ "_orig_mod.ctx_encoder.4.weight", "_orig_mod.ctx_encoder.4.bias", "_orig_mod.  
↪ ctx_to_img.weight", "_orig_mod.ctx_to_img.bias", "_orig_mod.ctx_pos_emb.  
↪ weight", "_orig_mod.ctx_time_emb.weight", "_orig_mod.unet.conv_in.weight",  
↪ "_orig_mod.unet.conv_in.bias", "_orig_mod.unet.time_embedding.linear_1.  
↪ weight", "_orig_mod.unet.time_embedding.linear_1.bias", "_orig_mod.unet.  
↪ time_embedding.linear_2.weight", "_orig_mod.unet.time_embedding.linear_2.  
↪ bias", "_orig_mod.unet.down_blocks.0.resnets.0.norm1.weight", "_orig_mod.unet  
↪ down_blocks.0.resnets.0.norm1.bias", "_orig_mod.unet.down_blocks.0.resnets.0.  
↪ conv1.weight", "_orig_mod.unet.down_blocks.0.resnets.0.conv1.bias", "_orig_mod.  
↪ unet.down_blocks.0.resnets.0.time_emb_proj.weight", "_orig_mod.unet.  
↪ down_blocks.0.resnets.0.time_emb_proj.bias", "_orig_mod.unet.down_blocks.0.  
↪ resnets.0.norm2.weight", "_orig_mod.unet.down_blocks.0.resnets.0.norm2.bias",  
↪ "_orig_mod.unet.down_blocks.0.resnets.0.conv2.weight", "_orig_mod.unet.  
↪ down_blocks.0.resnets.0.conv2.bias", "_orig_mod.unet.down_blocks.0.resnets.1.  
↪ norm1.weight", "_orig_mod.unet.down_blocks.0.resnets.1.norm1.bias", "_orig_mod.  
↪ unet.down_blocks.0.resnets.1.conv1.weight", "_orig_mod.unet.down_blocks.0.  
↪ resnets.1.conv1.bias", "_orig_mod.unet.down_blocks.0.resnets.1.time_emb_proj.  
↪ weight", "_orig_mod.unet.down_blocks.0.resnets.1.time_emb_proj.bias",  
↪ "_orig_mod.unet.down_blocks.0.resnets.1.norm2.weight", "_orig_mod.unet.  
↪ down_blocks.0.resnets.1.norm2.bias", "_orig_mod.unet.down_blocks.0.resnets.1.  
↪ conv2.weight", "_orig_mod.unet.down_blocks.0.resnets.1.conv2.bias", "_orig_mod.  
↪ unet.down_blocks.0.downsamplers.0.conv.weight", "_orig_mod.unet.down_blocks.0.  
↪ downsamplers.0.conv.bias", "_orig_mod.unet.down_blocks.1.resnets.0.norm1.  
↪ weight", "_orig_mod.unet.down_blocks.1.resnets.0.norm1.bias", "_orig_mod.unet  
↪ down_blocks.1.resnets.0.conv1.weight", "_orig_mod.unet.down_blocks.1.resnets.0.  
↪ conv1.bias", "_orig_mod.unet.down_blocks.1.resnets.0.time_emb_proj.weight",  
↪ "_orig_mod.unet.down_blocks.1.resnets.0.time_emb_proj.bias", "_orig_mod.unet.  
↪ down_blocks.1.resnets.0.norm2.weight", "_orig_mod.unet.down_blocks.1.resnets.0.  
↪ norm2.bias", "_orig_mod.unet.down_blocks.1.resnets.0.conv2.weight", "_orig_mod.  
↪ unet.down_blocks.1.resnets.0.conv2.bias", "_orig_mod.unet.down_blocks.1.  
↪ resnets.0.conv_shortcut.weight", "_orig_mod.unet.down_blocks.1.resnets.0.  
↪ conv_shortcut.bias", "_orig_mod.unet.down_blocks.1.resnets.1.norm1.weight",  
↪ "_orig_mod.unet.down_blocks.1.resnets.1.norm1.bias", "_orig_mod.unet.  
↪ down_blocks.1.resnets.1.conv1.weight", "_orig_mod.unet.down_blocks.1.resnets.1.  
↪ conv1.bias", "_orig_mod.unet.down_blocks.1.resnets.1.time_emb_proj.weight",  
↪ "_orig_mod.unet.down_blocks.1.resnets.1.time_emb_proj.bias", "_orig_mod.unet.  
↪ down_blocks.1.resnets.1.norm2.weight", "_orig_mod.unet.down_blocks.1.resnets.1.  
↪ norm2.bias", "_orig_mod.unet.down_blocks.1.resnets.1.conv2.weight", "_orig_mod.  
↪ unet.down_blocks.1.resnets.1.conv2.bias", "_orig_mod.unet.down_blocks.1.  
↪ downsamplers.0.conv.weight", "_orig_mod.unet.down_blocks.1.downsamplers.0.conv.  
↪ bias", "_orig_mod.unet.down_blocks.2.attentions.0.norm.weight", "_orig_mod.  
↪ unet.down_blocks.2.attentions.0.norm.bias", "_orig_mod.unet.down_blocks.2.  
↪ attentions.0.proj_in.weight", "_orig_mod.unet.down_blocks.2.attentions.0.  
↪ proj_in.bias", "_orig_mod.unet.down_blocks.2.attentions.0.transformer_blocks.0.  
↪ norm1.weight", "_orig_mod.unet.down_blocks.2.attentions.0.transformer_blocks.0.  
↪ norm1.bias", "_orig_mod.unet.down_blocks.2.attentions.0.transformer_blocks.0.  
↪ attn1.to_q.weight", "_orig_mod.unet.down_blocks.2.attentions.0.  
↪ transformer_blocks.0.attn1.to_k.weight", "_orig_mod.unet.down_blocks.2.  
↪ attentions.0.transformer_blocks.0.attn1.to_v.weight", "_orig_mod.unet.  
↪ down_blocks.2.attentions.0.transformer_blocks.0.attn1.to_out.0.weight",  
↪ "_orig_mod.unet.down_blocks.2.attentions.0.transformer_blocks.0.attn1.to_out.0.  
↪ bias", "_orig_mod.unet.down_blocks.2.attentions.0.transformer_blocks.0.norm2.  
↪ weight", "_orig_mod.unet.down_blocks.2.attentions.0.transformer_blocks.0.norm2.  
↪ bias", "_orig_mod.unet.down_blocks.2.attentions.0.transformer_blocks.0.attn2.  
↪ to_q.weight", "_orig_mod.unet.down_blocks.2.attentions.0.transformer_blocks.0.  
↪ attn2.to_k.weight", "_orig_mod.unet.down_blocks.2.attentions.0.  
↪ transformer_blocks.0.attn2.to_v.weight", "_orig_mod.unet.down_blocks.2.  
↪ attentions.0.transformer_blocks.0.attn2.to_out.0.weight", "_orig_mod.unet.  
↪ down_blocks.2.attentions.0.transformer_blocks.0.attn2.to_out.0.bias",  
↪ "_orig_mod.unet.down_blocks.2.attentions.0.transformer_blocks.0.norm3.weight",  
↪ "_orig_mod.unet.down_blocks.2.attentions.0.transformer_blocks.0.norm3.bias",  
↪ "_orig_mod.unet.down_blocks.2.attentions.0.transformer_blocks.0.ff.net.0.proj.  
↪ weight", "_orig_mod.unet.down_blocks.2.attentions.0.transformer_blocks.0.ff.  
↪ net.0.proj.bias", "_orig_mod.unet.down_blocks.2.attentions.0.  
↪ transformer_blocks.0.ff.net.2.weight", "_orig_mod.unet.down_blocks.2.  
↪ attentions.0.transformer_blocks.0.ff.net.2.bias", "_orig_mod.unet.down_blocks  
↪ 2.attentions.0.proj_out.weight", "_orig_mod.unet.down_blocks.2.attentions.0.  
↪ proj_out.bias", "_orig_mod.unet.down_blocks.2.attentions.1.norm.weight",  
↪ "_orig_mod.unet.down_blocks.2.attentions.1.norm.bias", "_orig_mod.unet.  
↪ down_blocks.2.attentions.1.proj_in.weight", "_orig_mod.unet.down_blocks.2.  
↪ attentions.1.proj_in.bias", "_orig_mod.unet.down_blocks.2.attentions.1.  
↪ transformer_blocks.0.norm1.weight", "_orig_mod.unet.down_blocks.2.attentions.1.  
↪ transformer_blocks.0.norm1.bias", "_orig_mod.unet.down_blocks.2.attentions.1.
```

1.3 2) Load the test dataset the same way

We rebuild the **frame index** from silver data (cached) and reuse the cached test split if available.

```
[ ]: # Phase labels (same order as training)
PHASE_LABELS = [
    'pPB2', 'pPNa', 'pPNf', 'p2', 'p3', 'p4', 'p5', 'p6', 'p7', 'p8', 'p9+', 'pM', 'pSB', 'pB', 'pEB', 'pHB']
NUM_PHASES = len(PHASE_LABELS)

INDEX_CACHE = PROJECT_ROOT / 'cache' / 'silver_all_planes_frame_index.csv'
INDEX_CACHE.parent.mkdir(parents=True, exist_ok=True)

def build_index_from_silver():
    rows = []
    for plane in FOCAL_PLANES:
        plane_root = SILVER_ROOT / plane
        if not plane_root.exists():
            continue
        embryo_ids = sorted([d.name for d in plane_root.iterdir() if d.is_dir()])
        for eid in tqdm(embryo_ids, desc=f'Indexing {plane}'):
            folder = plane_root / eid
            ann_path = folder / f'{eid}_phases.csv'
            if not ann_path.exists():
                continue
            img_files = sorted(glob.glob(str(folder / '*.jpeg'))) + glob.glob(str(folder / '*.jpg'))
            if not img_files:
                continue
            frame_numbers = {}
            for f in img_files:
                basename = os.path.basename(f)
                m = re.search(r'(\d+)\.jpe?g$', basename)
                if m:
                    frame_num = int(m.group(1))
                    frame_numbers[frame_num] = f
            df = pd.read_csv(ann_path, header=None,
names=['phase', 'start', 'end'], sep='[,\s]+', engine='python')
            def phase_to_id(ph: str):
                ph = ph.strip()
                ph = ph.replace('t', 'p', 1) if ph.startswith('t') else ph
                return PHASE_LABELS.index(ph)
            label_spans = []
            for _, r in df.iterrows():
                try:
                    pid = phase_to_id(r['phase'])
                    label_spans.append((int(r['start']), int(r['end']), pid))
                except Exception:
```

```

        pass
    for frame_num, path in frame_numbers.items():
        pid = None
        for s, e, p in label_spans:
            if s <= frame_num <= e:
                pid = p
                break
        if pid is None:
            continue
        rows.append({'embryo_id': eid, 'plane': plane, 'frame_num': frame_num,
        ↪ 'path': path, 'phase_id': pid})
    return pd.DataFrame(rows)

if INDEX_CACHE.exists():
    index_df = pd.read_csv(INDEX_CACHE)
else:
    index_df = build_index_from_silver()
    index_df.to_csv(INDEX_CACHE, index=False)
print('Indexed frames:', len(index_df))

# --- Splits (use cached from training if present) ---
SPLIT_SEED = 42
VAL_SPLIT = 0.15
TEST_SPLIT = 0.15
SPLIT_CACHE_STEM = (
    f'splits_embryo_phase_ctxk{CONTEXT_K}_'
    f'seed{SPLIT_SEED}_v{VAL_SPLIT}_t{TEST_SPLIT}'
)
CACHE_DIR = PROJECT_ROOT / 'cache'
TRAIN_SPLIT_CACHE = CACHE_DIR / f'{SPLIT_CACHE_STEM}_train.csv'
VAL_SPLIT_CACHE = CACHE_DIR / f'{SPLIT_CACHE_STEM}_val.csv'
TEST_SPLIT_CACHE = CACHE_DIR / f'{SPLIT_CACHE_STEM}_test.csv'

def stratified_split(ids: list[str], labels: dict[str, int], val_frac: float,
    ↪ test_frac: float, seed: int):
    rng = np.random.RandomState(seed)
    train_ids, val_ids, test_ids = [], [], []
    by_label: dict[int, list[str]] = {}
    for eid in ids:
        by_label.setdefault(labels[eid], []).append(eid)
    for label, group in by_label.items():
        group = list(group)
        rng.shuffle(group)
        n = len(group)
        n_test = int(round(n * test_frac))
        n_val = int(round(n * val_frac))
        n_test = min(n_test, n)

```

```

        n_val = min(n_val, n - n_test)
        test_ids.extend(group[:n_test])
        val_ids.extend(group[n_test:n_test + n_val])
        train_ids.extend(group[n_test + n_val:])
        return set(train_ids), set(val_ids), set(test_ids)

if TRAIN_SPLIT_CACHE.exists() and VAL_SPLIT_CACHE.exists() and TEST_SPLIT_CACHE.
    exists():
    train_df = pd.read_csv(TRAIN_SPLIT_CACHE)
    val_df = pd.read_csv(VAL_SPLIT_CACHE)
    test_df = pd.read_csv(TEST_SPLIT_CACHE)
else:
    embryo_phase = (
        index_df.groupby('embryo_id')['phase_id']
        .agg(lambda s: s.value_counts().idxmax())
        .to_dict()
    )
    embryo_ids = list(embryo_phase.keys())
    train_ids, val_ids, test_ids = stratified_split(embryo_ids, embryo_phase,
    VAL_SPLIT, TEST_SPLIT, SPLIT_SEED)
    train_df = pd.DataFrame({'embryo_id': list(train_ids)})
    val_df = pd.DataFrame({'embryo_id': list(val_ids)})
    test_df = pd.DataFrame({'embryo_id': list(test_ids)})
    train_df.to_csv(TRAIN_SPLIT_CACHE, index=False)
    val_df.to_csv(VAL_SPLIT_CACHE, index=False)
    test_df.to_csv(TEST_SPLIT_CACHE, index=False)

train_ids = set(train_df['embryo_id'].tolist())
val_ids = set(val_df['embryo_id'].tolist())
test_ids = set(test_df['embryo_id'].tolist())
print(f"Splits: train={len(train_ids)} val={len(val_ids)} test={len(test_ids)}")

# --- Dataset helpers ---
def _preprocess_pil(img: Image.Image, size: int = IMG_SIZE) -> Image.Image:
    w, h = img.size
    m = min(w, h)
    left = (w - m) // 2
    top = (h - m) // 2
    img = img.crop((left, top, left + m, top + m))
    if size is not None and (img.size[0] != size or img.size[1] != size):
        img = img.resize((size, size), Image.BICUBIC)
    return img
def load_pil(path: str, size: int = IMG_SIZE) -> Image.Image:
    img = Image.open(path).convert('L')
    return _preprocess_pil(img, size=size)
def pil_to_tensor(img: Image.Image) -> torch.Tensor:
    arr = np.array(img, dtype=np.float32) / 255.0

```

```

    arr = arr * 2 - 1
    return torch.from_numpy(arr).unsqueeze(0)
def load_image(path: str, size: int = IMG_SIZE):
    return pil_to_tensor(load_pil(path, size=size))
def denorm(x):
    return (x.clamp(-1,1) + 1) / 2

class PairDataset:
    """Windowed next-frame pairs (strict adjacency)."""
    def __init__(self, index_df: pd.DataFrame, embryo_ids: set[str],
↳ build_transition_index: bool = True):
        self.df = index_df[index_df['embryo_id'].isin(set(embryo_ids))].copy()
↳ reset_index(drop=True)
        self.build_transition_index = build_transition_index
        self.sequences: list[tuple[str, str]] = []
        self.frames_by_seq: dict[tuple[str, str], list[int]] = {}
        self.path_by_seq: dict[tuple[str, str], dict[int, str]] = {}
        self.phase_by_seq: dict[tuple[str, str], dict[int, int]] = {}
        self.valid_t_ends_by_seq: dict[tuple[str, str], list[int]] = {}
        self.transition_t_ends_by_seq: dict[tuple[str, str], list[int]] = {}
        self.non_transition_t_ends_by_seq: dict[tuple[str, str], list[int]] = {}
        self.windows: list[dict] = []
        self.windows_by_plane: dict[str, list[dict]] = {}
        self.transition_windows_by_plane: dict[str, list[dict]] = {}
        self.non_transition_windows_by_plane: dict[str, list[dict]] = {}
        self._rollout_candidates_by_steps: dict[int, list[dict]] = {}
        self._build_index()
    def _build_index(self):
        for (eid, plane), g in self.df.groupby(['embryo_id', 'plane']):
            g = g.sort_values('frame_num').reset_index(drop=True)
            frame_nums = g['frame_num'].tolist()
            if len(frame_nums) <= CONTEXT_K:
                continue
            frame_set = set(frame_nums)
            path_by = dict(zip(frame_nums, g['path'].tolist()))
            phase_by = dict(zip(frame_nums, g['phase_id'].tolist()))
            valid_t_ends = []
            for t_end in frame_nums:
                if (t_end + 1) not in frame_set:
                    continue
                ok = True
                for f in range(t_end - CONTEXT_K + 1, t_end + 1):
                    if f not in frame_set:
                        ok = False
                        break
                if ok:
                    valid_t_ends.append(t_end)

```

```

        if not valid_t_ends:
            continue
        transition_t_ends = []
        non_transition_t_ends = []
        if self.build_transition_index:
            transition_t_ends = [t for t in valid_t_ends if phase_by.get(t)
↪ != phase_by.get(t + 1)]
            non_transition_t_ends = [t for t in valid_t_ends if phase_by.
↪ get(t) == phase_by.get(t + 1)]
            seq = (eid, plane)
            self.sequences.append(seq)
            self.frames_by_seq[seq] = frame_nums
            self.path_by_seq[seq] = path_by
            self.phase_by_seq[seq] = phase_by
            self.valid_t_ends_by_seq[seq] = valid_t_ends
            self.transition_t_ends_by_seq[seq] = transition_t_ends
            self.non_transition_t_ends_by_seq[seq] = non_transition_t_ends
            for t_end in valid_t_ends:
                row = {
                    'embryo_id': eid,
                    'plane': plane,
                    't_end': t_end,
                    'tgt_frame': t_end + 1,
                }
                self.windows.append(row)
                self.windows_by_plane.setdefault(plane, []).append(row)
                if self.build_transition_index and t_end in transition_t_ends:
                    self.transition_windows_by_plane.setdefault(plane, []).
↪ append(row)
                elif self.build_transition_index:
                    self.non_transition_windows_by_plane.setdefault(plane, []).
↪ append(row)
            def _build_row(self, row: dict):
                seq = (row['embryo_id'], row['plane'])
                t_end = row['t_end']
                ctx_frames = [t_end - i for i in range(CONTEXT_K - 1, -1, -1)]
                ctx_paths = [self.path_by_seq[seq][f] for f in ctx_frames]
                tgt_path = self.path_by_seq[seq][t_end + 1]
                out = dict(row)
                out['ctx_paths'] = ctx_paths
                out['tgt_path'] = tgt_path
                out['phase_id_tgt'] = self.phase_by_seq[seq].get(t_end + 1)
                out['is_transition'] = (self.phase_by_seq[seq].get(t_end) != self.
↪ phase_by_seq[seq].get(t_end + 1))
                return out
            def sample_rows(self, n: int, mode: str = 'any'):
                rows = []

```

```

rng = np.random.RandomState(SEED)
planes = list(self.windows_by_plane.keys())
for plane in planes:
    if len(rows) >= n:
        break
    if mode == 'transition':
        pool = self.transition_windows_by_plane.get(plane, [])
    elif mode == 'non_transition':
        pool = self.non_transition_windows_by_plane.get(plane, [])
    else:
        pool = self.windows_by_plane.get(plane, [])
    if not pool:
        continue
    picks = [pool[rng.randint(len(pool))] for _ in range(min(n -
↳ len(rows), len(pool)))]
    rows.extend([self._build_row(r) for r in picks])
return rows

def _build_rollout_candidates(self, steps: int):
    candidates = []
    for seq in self.frames_by_seq.keys():
        frame_nums = self.frames_by_seq[seq]
        frame_set = set(frame_nums)
        for t_end in self.valid_t_ends_by_seq.get(seq, []):
            ok = True
            for k in range(1, steps + 1):
                if (t_end + k) not in frame_set:
                    ok = False
                    break
            if ok:
                row = {'embryo_id': seq[0], 'plane': seq[1], 't_end': t_end}
                candidates.append(self._build_row(row))
    return candidates

def get_rollout_seed(self, steps: int, rng: np.random.RandomState):
    if steps not in self._rollout_candidates_by_steps:
        self._rollout_candidates_by_steps[steps] = self.
↳ _build_rollout_candidates(steps)
    candidates = self._rollout_candidates_by_steps[steps]
    if not candidates:
        return None
    # Choose a random phase, then a random embryo within that phase
    phase_ids = sorted({r['phase_id_tgt'] for r in candidates if r.
↳ get('phase_id_tgt') is not None})
    if not phase_ids:
        return rng.choice(candidates)
    target_phase = rng.choice(phase_ids)
    phase_cands = [r for r in candidates if r.get('phase_id_tgt') ==
↳ target_phase]

```

```

embryo_ids = sorted({r['embryo_id'] for r in phase_cands})
if not embryo_ids:
    return rng.choice(phase_cands)
target_embryo = rng.choice(embryo_ids)
embryo_cands = [r for r in phase_cands if r['embryo_id'] ==
↳target_embryo]
return rng.choice(embryo_cands)

test_dataset = PairDataset(index_df, test_ids, build_transition_index=True)
print('Test windows:', len(test_dataset.windows))

```

Indexed frames: 289675

Splits: train=491 val=106 test=106

Test windows: 43115

1.4 3) Define evaluation protocols (experiments)

We compute quantitative metrics for: - **A**) One-step prediction (teacher-forced) - **B**) Rollout prediction (free-running) - **C**) Transition vs non-transition performance + per-phase breakdown

```

[ ]: # Optional LPIPS (if installed)
try:
    from lpips import LPIPS
    lpips_metric = LPIPS(net='alex').to(device)
    lpips_metric.eval()
    for p in lpips_metric.parameters():
        p.requires_grad_(False)
    print('LPIPS enabled')
except Exception as exc:
    lpips_metric = None
    print('LPIPS disabled:', exc)

# SSIM helper (operates on [-1,1] tensors by mapping to [0,1])
_gaussian_kernel_cache = {}
def _gaussian_kernel(window_size: int, sigma: float, device: torch.device,
↳dtype: torch.dtype):
    key = (window_size, sigma, device.type, device.index, dtype)
    if key in _gaussian_kernel_cache:
        return _gaussian_kernel_cache[key]
    coords = torch.arange(window_size, device=device, dtype=dtype) -
↳window_size // 2
    gauss = torch.exp(-(coords ** 2) / (2 * sigma ** 2))
    gauss /= gauss.sum()
    kernel_1d = gauss.unsqueeze(0)
    kernel_2d = kernel_1d.t() @ kernel_1d
    kernel_2d /= kernel_2d.sum()
    kernel = kernel_2d.unsqueeze(0).unsqueeze(0)
    _gaussian_kernel_cache[key] = kernel

```

```

    return kernel
def compute_ssim(pred: torch.Tensor, target: torch.Tensor, window_size: int = 11, sigma: float = 1.5) -> torch.Tensor:
    pred = pred.clamp(-1, 1)
    target = target.clamp(-1, 1)
    pred_01 = (pred + 1) / 2
    target_01 = (target + 1) / 2
    device = pred.device
    dtype = pred.dtype
    channels = pred_01.size(1)
    kernel = _gaussian_kernel(window_size, sigma, device, dtype).
    expand(channels, 1, window_size, window_size)
    padding = window_size // 2
    mu_pred = F.conv2d(pred_01, kernel, padding=padding, groups=channels)
    mu_target = F.conv2d(target_01, kernel, padding=padding, groups=channels)
    mu_pred_sq = mu_pred.pow(2)
    mu_target_sq = mu_target.pow(2)
    mu_pred_target = mu_pred * mu_target
    sigma_pred = F.conv2d(pred_01 * pred_01, kernel, padding=padding,
    groups=channels) - mu_pred_sq
    sigma_target = F.conv2d(target_01 * target_01, kernel, padding=padding,
    groups=channels) - mu_target_sq
    sigma_pred_target = F.conv2d(pred_01 * target_01, kernel, padding=padding,
    groups=channels) - mu_pred_target
    sigma_pred = torch.clamp_min(sigma_pred, 0.0)
    sigma_target = torch.clamp_min(sigma_target, 0.0)
    c1 = (0.01) ** 2
    c2 = (0.03) ** 2
    numerator = (2 * mu_pred_target + c1) * (2 * sigma_pred_target + c2)
    denominator = (mu_pred_sq + mu_target_sq + c1) * (sigma_pred + sigma_target
    + c2)
    return (numerator / (denominator + 1e-6)).mean()

def compute_lpips(pred: torch.Tensor, target: torch.Tensor) -> float | None:
    if lpips_metric is None:
        return None
    # LPIPS expects 3 channels
    pred_in = pred.repeat(1, 3, 1, 1).clamp(-1, 1)
    target_in = target.repeat(1, 3, 1, 1).clamp(-1, 1)
    with torch.no_grad():
        return float(lpips_metric(pred_in, target_in, normalize=False).mean())

def _rows_to_tensors(rows: list[dict]):
    ctx_list, tgt_list, frame_idx_list = [], [], []
    for row in rows:
        ctx_tensors = [load_image(p).squeeze(0) for p in row['ctx_paths']]

```

```

        x_ctx = torch.stack(ctx_tensors, dim=0)
        x_tgt = load_image(row['tgt_path'])
        ctx_list.append(x_ctx)
        tgt_list.append(x_tgt)
        tgt_frame = row.get('tgt_frame', int(row['t_end']) + 1)
        frame_idx_list.append(min(int(tgt_frame), int(MAX_FRAME_INDEX)))
    x_ctx = torch.stack(ctx_list, dim=0)
    x_tgt = torch.stack(tgt_list, dim=0)
    frame_idx = torch.tensor(frame_idx_list, dtype=torch.long)
    return x_ctx, x_tgt, frame_idx

@torch.no_grad()
def eval_one_step(rows: list[dict], desc: str):
    metrics = []
    for i in tqdm(range(0, len(rows), 16), desc=desc):
        batch = rows[i:i+16]
        x_ctx, x_tgt, frame_idx = _rows_to_tensors(batch)
        x_ctx = x_ctx.to(device)
        x_tgt = x_tgt.to(device)
        frame_idx = frame_idx.to(device)
        x_gen = sample_from_model((x_ctx.size(0), 1, IMG_SIZE, IMG_SIZE),
        ↪x_ctx=x_ctx, frame_idx=frame_idx)
        mse = F.mse_loss(x_gen, x_tgt).item()
        mae = F.l1_loss(x_gen, x_tgt).item()
        ssim = compute_ssim(x_gen, x_tgt).item()
        lpips_val = compute_lpips(x_gen, x_tgt)
        metrics.append({'mse': mse, 'mae': mae, 'ssim': ssim, 'lpips':
        ↪lpips_val})
    return metrics

@torch.no_grad()
def eval_rollout(dataset: PairDataset, num_samples: int, steps: int):
    results = []
    rng = np.random.RandomState(SEED + 9001)
    for _ in tqdm(range(num_samples), desc='Rollout eval'):
        seed_row = dataset.get_rollout_seed(steps, rng=rng)
        if seed_row is None:
            break
        # initial context
        ctx_tensors = [load_image(p).squeeze(0) for p in seed_row['ctx_paths']]
        x_ctx = torch.stack(ctx_tensors, dim=0).unsqueeze(0).to(device)
        base_t_end = int(seed_row['t_end'])
        for k in range(1, steps + 1):
            frame_idx = torch.tensor([min(base_t_end + k, MAX_FRAME_INDEX)],
            ↪device=device, dtype=torch.long)
            x_gen = sample_from_model((1, 1, IMG_SIZE, IMG_SIZE), x_ctx=x_ctx,
            ↪frame_idx=frame_idx)

```

```

        # ground truth target at base_t_end + k
        tgt_path = dataset.path_by_seq[(seed_row['embryo_id'],
↳seed_row['plane'])][base_t_end + k]
        x_tgt = load_image(tgt_path).unsqueeze(0).to(device)
        mse = F.mse_loss(x_gen, x_tgt).item()
        mae = F.l1_loss(x_gen, x_tgt).item()
        ssim = compute_ssim(x_gen, x_tgt).item()
        lpips_val = compute_lpips(x_gen, x_tgt)
        results.append({'step': k, 'mse': mse, 'mae': mae, 'ssim': ssim,
↳'lpips': lpips_val})
        # update context window
        x_ctx = torch.cat([x_ctx[:, 1:], x_gen.detach()], dim=1)
    return results

```

Setting up [LPIPS] perceptual loss: trunk [alex], v[0.1], spatial [off]

/home/khasion/miniconda3/envs/thesis/lib/python3.10/site-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.

```

warnings.warn(
/home/khasion/miniconda3/envs/thesis/lib/python3.10/site-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=AlexNet_Weights.IMAGENET1K_V1`. You can also use `weights=AlexNet_Weights.DEFAULT` to get the most up-to-date weights.
warnings.warn(msg)

```

Loading model from: /home/khasion/miniconda3/envs/thesis/lib/python3.10/site-packages/lpips/weights/v0.1/alex.pth
LPIPS enabled

1.5 4) Generate qualitative grids

We save a **one-step grid** and a **rollout grid**. Each grid shows all context frames, then targets, then generated frames.

```

[ ]: def _plot_one_step_grid(row: dict, out_path: Path):
    ctx_tensors = [load_image(p).squeeze(0) for p in row['ctx_paths']]
    x_ctx = torch.stack(ctx_tensors, dim=0).unsqueeze(0).to(device)
    tgt_frame = row.get('tgt_frame', int(row['t_end']) + 1)
    frame_idx = torch.tensor([min(int(tgt_frame), MAX_FRAME_INDEX)],
↳device=device, dtype=torch.long)
    x_gen = sample_from_model((1, 1, IMG_SIZE, IMG_SIZE), x_ctx=x_ctx,
↳frame_idx=frame_idx)
    x_tgt = load_image(row['tgt_path'])

    ncols = CONTEXT_K + 2

```

```

fig, axes = plt.subplots(1, ncols, figsize=(3 * ncols, 3))
if ncols == 1:
    axes = [axes]

# Context frames
for i, ctx in enumerate(ctx_tensors):
    axes[i].imshow(denorm(ctx.unsqueeze(0))[0].cpu().numpy(), cmap='gray')
    axes[i].set_title(f"Ctx (t-{CONTEXT_K - i})")
    axes[i].axis('off')

# Target
axes[CONTEXT_K].imshow(denorm(x_tgt)[0].cpu().numpy(), cmap='gray')
axes[CONTEXT_K].set_title('Target (t+1)')
axes[CONTEXT_K].axis('off')

# Generated
axes[CONTEXT_K + 1].imshow(denorm(x_gen[0])[0].cpu().numpy(), cmap='gray')
axes[CONTEXT_K + 1].set_title('Generated')
axes[CONTEXT_K + 1].axis('off')

fig.tight_layout()
fig.savefig(out_path, dpi=150)
plt.close(fig)

def _plot_rollout_grid(row: dict, out_path: Path, steps: int):
    ctx_tensors = [load_image(p).squeeze(0) for p in row['ctx_paths']]
    x_ctx = torch.stack(ctx_tensors, dim=0).unsqueeze(0).to(device)
    base_t_end = int(row['t_end'])

    targets = []
    generated = []
    for k in range(1, steps + 1):
        frame_idx = torch.tensor([min(base_t_end + k, MAX_FRAME_INDEX)],
    ↪device=device, dtype=torch.long)
        x_gen = sample_from_model((1, 1, IMG_SIZE, IMG_SIZE), x_ctx=x_ctx,
    ↪frame_idx=frame_idx)
        tgt_path = test_dataset.path_by_seq[(row['embryo_id'],
    ↪row['plane'])][base_t_end + k]
        x_tgt = load_image(tgt_path)
        targets.append(x_tgt.squeeze(0))
        generated.append(x_gen[0].detach().cpu().squeeze(0))
        x_ctx = torch.cat([x_ctx[:, 1:], x_gen.detach()], dim=1)

    ncols = max(CONTEXT_K, steps)
    fig, axes = plt.subplots(3, ncols, figsize=(3 * ncols, 9))

    # Row 1: context

```

```

    for i in range(ncols):
        axes[0, i].axis('off')
    for i, ctx in enumerate(ctx_tensors):
        axes[0, i].imshow(denorm(ctx.unsqueeze(0))[0].cpu().numpy(),
        cmap='gray')
        axes[0, i].set_title(f"Ctx (t-{CONTEXT_K - i})")
        axes[0, i].axis('off')

    # Row 2: targets
    for i in range(ncols):
        axes[1, i].axis('off')
    for i, tgt in enumerate(targets):
        axes[1, i].imshow(denorm(tgt.unsqueeze(0))[0].cpu().numpy(),
        cmap='gray')
        axes[1, i].set_title(f"Target (t+{i+1})")
        axes[1, i].axis('off')

    # Row 3: generated
    for i in range(ncols):
        axes[2, i].axis('off')
    for i, gen in enumerate(generated):
        axes[2, i].imshow(denorm(gen.unsqueeze(0))[0].cpu().numpy(),
        cmap='gray')
        axes[2, i].set_title(f"Gen (t+{i+1})")
        axes[2, i].axis('off')

    fig.tight_layout()
    fig.savefig(out_path, dpi=150)
    plt.close(fig)

# Generate qualitative grids (multiple examples)
N_QUAL = 5
one_step_rows = test_dataset.sample_rows(n=N_QUAL, mode='any')
for i, row in enumerate(one_step_rows):
    _plot_one_step_grid(row, ARTIFACT_DIR / f'one_step_grid_{i+1:02d}.png')

# Rollout grids (random embryo + random phase each time)
rng = np.random.RandomState(SEED + 1337)
for i in range(N_QUAL):
    rollout_row = test_dataset.get_rollout_seed(steps=ROLLOUT_STEPS, rng=rng)
    if rollout_row is None:
        break
    _plot_rollout_grid(rollout_row, ARTIFACT_DIR / f'rollout_grid_{i+1:02d}.
    png', steps=ROLLOUT_STEPS)

print('Saved grids to:', ARTIFACT_DIR)

```

Saved grids to:

/home/khasion/Projects/thesis/diffusion/diffusion_samples/eval_artifacts

1.6 5) Run evaluation and save artifacts

We run all protocols and write a single CSV file: `test__metrics.csv`.

```
[ ]: # A) One-step prediction (teacher-forced)
one_step_rows = test_dataset.sample_rows(n=EVAL_MAX_SAMPLES, mode='any')
one_step_metrics = eval_one_step(one_step_rows, desc='One-step eval')

# B) Rollout prediction (free-running)
rollout_metrics = eval_rollout(test_dataset, num_samples=ROLLOUT_SAMPLES,
    ↪ steps=ROLLOUT_STEPS)

# C) Transition vs non-transition + per-phase breakdown
trans_rows = test_dataset.sample_rows(n=EVAL_MAX_SAMPLES, mode='transition')
non_rows = test_dataset.sample_rows(n=EVAL_MAX_SAMPLES, mode='non_transition')
trans_metrics = eval_one_step(trans_rows, desc='Transition eval')
non_metrics = eval_one_step(non_rows, desc='Non-transition eval')

# Per-phase breakdown (one-step only)
phase_records = []
for row in one_step_rows:
    pid = row.get('phase_id_tgt')
    if pid is None:
        continue
    phase_records.append(pid)
phase_records = np.array(phase_records) if len(phase_records) else None

def _aggregate(metrics: list[dict], label: str):
    if not metrics:
        return None
    df = pd.DataFrame(metrics)
    out = {
        'group': label,
        'n': len(df),
        'mse': df['mse'].mean(),
        'mae': df['mae'].mean(),
        'ssim': df['ssim'].mean(),
        'lpips': df['lpips'].dropna().mean() if 'lpips' in df and df['lpips'].
    ↪ notna().any() else np.nan,
    }
    return out

summary_rows = []
for label, metrics in [
    ('one_step_all', one_step_metrics),
```

```

        ('one_step_transition', trans_metrics),
        ('one_step_non_transition', non_metrics),
]:
    agg = _aggregate(metrics, label)
    if agg is not None:
        summary_rows.append(agg)

# Rollout summary by step
if rollout_metrics:
    rollout_df = pd.DataFrame(rollout_metrics)
    for step, g in rollout_df.groupby('step'):
        summary_rows.append({
            'group': f'rollout_step_{int(step)}',
            'n': len(g),
            'mse': g['mse'].mean(),
            'mae': g['mae'].mean(),
            'ssim': g['ssim'].mean(),
            'lpips': g['lpips'].dropna().mean() if g['lpips'].notna().any()
        }
        else np.nan,
        })

# Per-phase summary (one-step)
if phase_records is not None and len(phase_records) == len(one_step_metrics):
    df_one = pd.DataFrame(one_step_metrics)
    df_one['phase_id'] = phase_records
    for pid, g in df_one.groupby('phase_id'):
        summary_rows.append({
            'group': f'phase_{PHASE_LABELS[int(pid)]}',
            'n': len(g),
            'mse': g['mse'].mean(),
            'mae': g['mae'].mean(),
            'ssim': g['ssim'].mean(),
            'lpips': g['lpips'].dropna().mean() if g['lpips'].notna().any()
        }
        else np.nan,
        })

# Save artifacts
summary_df = pd.DataFrame(summary_rows)
out_csv = ARTIFACT_DIR / 'test_metrics.csv'
summary_df.to_csv(out_csv, index=False)
print('Saved:', out_csv)
summary_df.head(10)

```

```

One-step eval: 100%|          | 16/16 [04:02<00:00, 15.15s/it]
Rollout eval: 100%|          | 12/12 [03:32<00:00, 17.71s/it]
Transition eval: 100%|        | 16/16 [04:03<00:00, 15.24s/it]
Non-transition eval: 100%|     | 16/16 [04:04<00:00, 15.27s/it]

```

Saved: /home/khasion/Projects/thesis/diffusion/diffusion_samples/eval_artifacts/
test_metrics.csv

```
[ ]:
      group  n      mse      mae      ssim      lpips
0      one_step_all  16  0.184962  0.329564  0.401132  0.233184
1      one_step_transition  16  0.178904  0.322900  0.410748  0.217409
2  one_step_non_transition  16  0.188848  0.332579  0.397360  0.207183
3      rollout_step_1  12  0.154253  0.284589  0.474217  0.173992
4      rollout_step_2  12  0.172191  0.297356  0.393165  0.216538
5      rollout_step_3  12  0.187600  0.326592  0.371655  0.222270
6      rollout_step_4  12  0.177779  0.298028  0.374416  0.223798
7      rollout_step_5  12  0.212664  0.346292  0.251052  0.370739
8      rollout_step_6  12  0.230335  0.359509  0.267488  0.290965
9      rollout_step_7  12  0.219777  0.350345  0.287494  0.308194
```